TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



KIỂM THỬ PHẦN MỀM

Assignment 1 - Kiểm Thử Hộp Đen

Thiết kế test case và tự động hóa web với JUnit 5

(Version 1.0)

GVHD: Từ Lãng Phiêu

TP. HỒ CHÍ MINH, THÁNG 10/2026

Mục lục

1 Tổng quan: 3

2 Hệ thống kiểm thử và mã nguồn mẫu 3

2.1 SauceDemo . . . 3
2.2 Cấu trúc mã nguồn mẫu . . . 5
2.3 Mã nguồn sinh viên phải bổ sung . . . 6

3 NV1 – Chạy và mở rộng mã mẫu 6

4 NV2 – Phân tích yêu cầu 6

4.1 Đặc tả yêu cầu . . . 6
4.2 Đánh giá rủi ro . . . 7
4.3 Ma trận truy vết . . . 7

5 NV3 – Thiết kế test case 7

5.1 Phân lớp tương đương (EP) . . . 7
5.2 Phân tích giá trị biên (BVA) . . . 8
5.3 Bảng quyết định (DT) . . . 8
5.4 Kiểm thử ca sử dụng (UC) . . . 9
5.5 Số lượng tối thiểu . . . 11

6 NV4 – Thực thi thủ công và báo lỗi 12

6.1 Ghi kết quả thực thi . . . 12
6.2 Báo cáo lỗi . . . 12

7 NV5 – Tự động hóa bằng JUnit 5 12

7.1 Yêu cầu . . . 12
7.2 Ví dụ Page Object . . . 13
7.3 Ví dụ test tham số hóa từ kịch bản use case . . . 13
7.4 Ví dụ kiểm tra sắp xếp (FR-04) . . . 14
7.5 Ví dụ đo NFR-01 và đánh dấu lỗi đã biết . . . 14
7.6 Chờ đúng cách . . . 15
7.7 Điểm bonus . . . 15

8 Sản phẩm nộp 15

9 Tự kiểm tra trước khi nộp 15

Hệ thống kiểm thử	SauceDemo – https://www.saucedemo.com
Công nghệ bắt buộc	Java 17+, JUnit 5 (Jupiter) 5.10+, Selenium WebDriver 4+, Maven
Mã nguồn mẫu	Thư mục `starter-code/` đi kèm đề bài
Hình thức	Nhóm <= 5 sinh viên
Sản phẩm nộp	Báo cáo PDF, tệp Excel test case, mã nguồn Maven

1 Tổng quan:

Nhóm kiểm thử bốn chức năng và một yêu cầu phi chức năng của SauceDemo – một trang bán hàng mẫu dành riêng cho việc kiểm thử. Bài làm gồm năm nhiệm vụ nối tiếp nhau. Kết quả của nhiệm vụ trước là đầu vào của nhiệm vụ sau:

NV1. **Chạy và mở rộng mã mẫu**: hiểu cấu trúc dự án, chạy được `mvn clean test`, thêm dữ liệu kiểm thử mới.

NV2. **Phân tích yêu cầu**: viết đặc tả FR-01 đến FR-04, NFR-01; lập ma trận truy vết và đánh giá rủi ro.

NV3. **Thiết kế test case** bằng phân lớp tương đương (EP), phân tích giá trị biên (BVA), bảng quyết định (DT) và kiểm thử ca sử dụng (UC).

NV4. **Thực thi thủ công và báo lỗi**: chạy test case, ghi kết quả thực tế, lập báo cáo lỗi.

NV5. **Tự động hóa bằng JUnit 5**: viết Page Object và test tham số hóa cho các test case đã thiết kế.

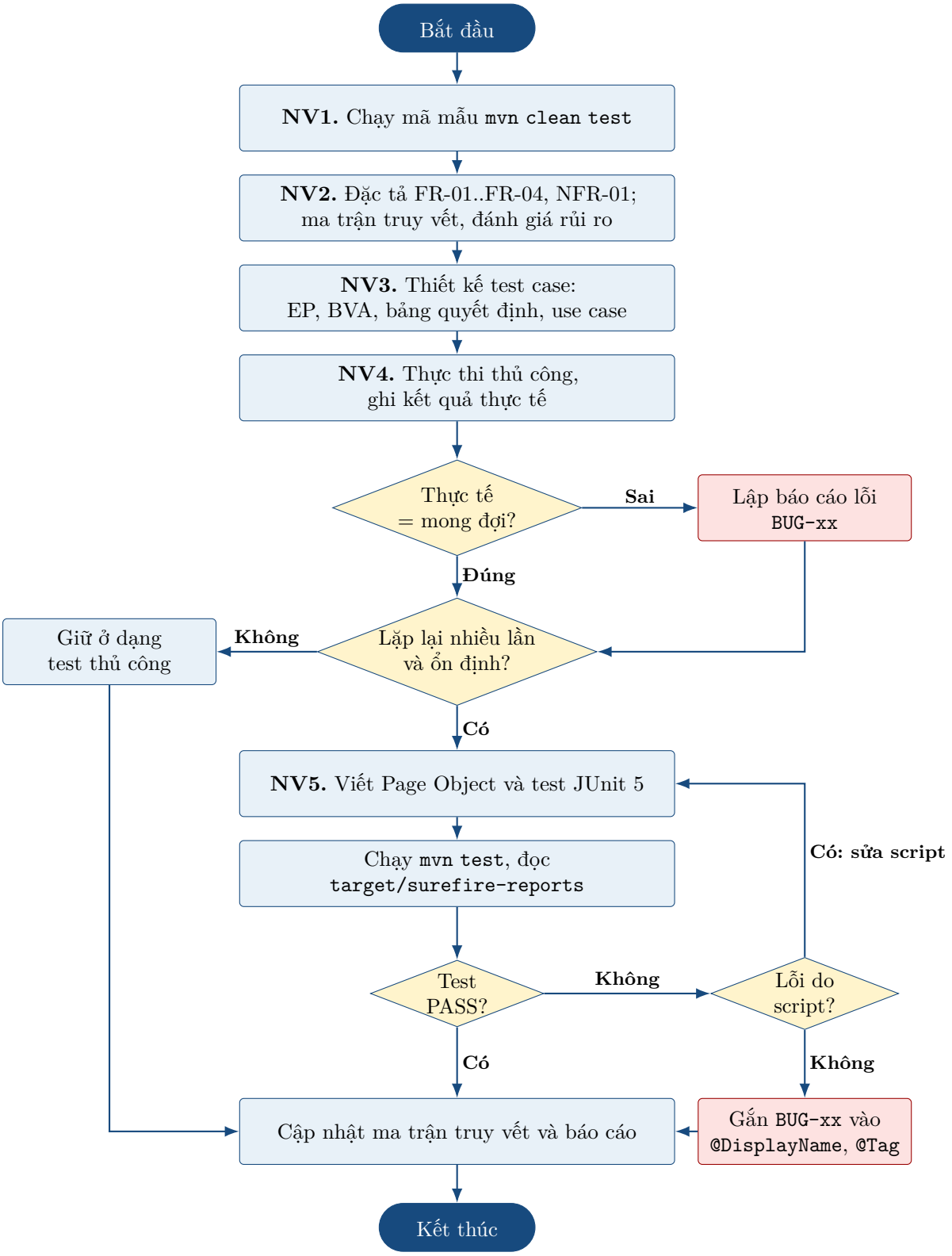
Hình 1 mô tả thứ tự công việc và các điểm ra quyết định. Lưu ý hai nhánh quan trọng: test case cho kết quả sai được ghi thành lỗi (BUG) rồi *vẫn* được xét tự động hóa; test tự động bị FAIL phải được phân loại là lỗi của script (sửa script) hay lỗi của hệ thống (gắn mã BUG), không được sửa assertion để test chuyển sang PASS.

2 Hệ thống kiểm thử và mã nguồn mẫu

2.1 SauceDemo

SauceDemo có trang đăng nhập, trang danh sách 6 sản phẩm (`inventory.html`), giỏ hàng (`cart.html`) và quy trình thanh toán gồm ba trang: nhập thông tin (`checkout-step-one.html`), tổng quan (`checkout-step-two.html`) và hoàn tất (`checkout-complete.html`). Mọi tài khoản dùng chung mật khẩu `secret_sauce`.

Tài khoản	Hành vi dùng để kiểm thử
`standard_user`	Hoạt động bình thường; dùng cho luồng chính.
`locked_out_user`	Bị khóa; đăng nhập đúng mật khẩu vẫn bị từ chối.
`problem_user`	Có lỗi giao diện được cài sẵn; dùng để tìm lỗi chức năng.
`performance_glitch_user`	Bị làm chậm có chủ đích; dùng cho NFR-01.



Hình 1: Quy trình thực hiện Assignment 1. Ô đỏ là các điểm phát sinh hoặc ghi nhận lỗi của hệ thống.

2.2 Cấu trúc mã nguồn mẫu

```
starter-code/
|-- pom.xml                          # JUnit 5.11, Selenium 4.27, Surefire 3.5
|-- README.md
‘-- src/test/
    |-- java/vn/edu/softwaretesting/
    |   |-- core/WebTestBase.java    # mo/dong trinh duyet cho moi test
    |   |-- pages/LoginPage.java     # Page Object cua trang dang nhap
    |   ‘-- tests/LoginTest.java     # test tham so hoa doc du lieu CSV
    ‘-- resources/
        ‘-- login-data.csv           # du lieu cho LoginTest
```

Listing 1: Cấu trúc thư mục `starter-code/`

Thành phần	Vai trò
`WebTestBase`	`@BeforeEach` tạo `WebDriver` mới, `@AfterEach` gọi `driver.quit()`. Nhờ vậy mỗi test chạy trên phiên trình duyệt sạch và không phụ thuộc nhau. Đọc `baseUrl`, `browser`, `headless` từ system property.
`LoginPage`	Chứa locator (`By.id("user-name")`, ...) và thao tác `open()`, `login()`. Hàm `login()` dùng explicit wait chờ trang sản phẩm *hoặc* thông báo lỗi xuất hiện, rồi trả về `LoginResult`. Test không truy cập locator trực tiếp.
`LoginTest`	Một phương thức `@ParameterizedTest` chạy một lần cho mỗi dòng của `login-data.csv`. Assertion so sánh kết quả thực tế với cột `accepted` và `expectedEvidence`.
`login-data.csv`	Mỗi dòng là một test case: username, password, kết quả mong đợi, đoạn văn bản/URL làm bằng chứng.

Cách chạy và kết quả mong đợi khi chưa sửa gì:

```
cd starter-code
mvn clean test                                  # Chrome, headless
mvn test -Dheadless=false                       # xem trinh duyet chay
mvn test -Dbrowser=firefox                      # doi trinh duyet

[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

2.3 Mã nguồn sinh viên phải bổ sung

Tệp cần tạo	Yêu cầu	Nội dung tối thiểu
`pages/InventoryPage`	FR-02, FR-04	Thêm/xóa sản phẩm, đọc số trên badge giỏ hàng, chọn kiểu sắp xếp, đọc danh sách tên/giá.
`pages/CartPage`	FR-02, FR-03	Đọc sản phẩm trong giỏ, nhấn Checkout.
`pages/CheckoutPage`	FR-03	Nhập thông tin, Continue, Cancel, đọc lỗi, đọc Item total/Tax/Total, Finish.
`tests/LoginTest`	FR-01	Mở rộng dữ liệu CSV theo bảng EP và bảng quyết định.
`tests/CartTest`	FR-02	Các giá trị biên của số lượng sản phẩm.
`tests/CheckoutTest`	FR-03	Toàn bộ kịch bản use case thanh toán.
`tests/SortTest`	FR-04	Bốn kiểu sắp xếp.
`tests/PerformanceTest`	NFR-01	Đo thời gian đăng nhập.

3 NV1 – Chạy và mở rộng mã mẫu

Việc cần làm. Chạy được dự án, sau đó thêm vào `login-data.csv` hai test case mới và chạy lại.

Ví dụ. Thêm test case "username rỗng":

```
username,password,accepted,expectedEvidence
standard_user,secret_sauce,true,inventory.html
locked_out_user,secret_sauce,false,locked out
standard_user,wrong_password,false,Username and password do not match
"",secret_sauce,false,Username is required
```

Chú ý: trong `@CsvFileSource`, ô để trống không có dấu nháy (`,secret_sauce`) được JUnit hiểu là `null`, khiến `sendKeys(null)` ném ngoại lệ. Muốn truyền chuỗi rỗng phải viết `""`. Sau khi thêm, kết quả phải là `Tests run: 4` và tên test hiển thị `TC-LOGIN-4: user=, accepted=false`.

Nộp trong báo cáo: ảnh chụp kết quả `mvn clean test`, phiên bản Java/Chrome đã dùng, giải thích ngắn vai trò của `@BeforeEach`, `@ParameterizedTest`, `@CsvFileSource`.

4 NV2 – Phân tích yêu cầu

4.1 Đặc tả yêu cầu

Nhóm viết đặc tả theo mẫu dưới đây. Mỗi yêu cầu phải có quy tắc kiểm tra được (có giá trị, thông báo hoặc URL cụ thể), không viết chung chung kiểu "hệ thống xử lý đúng".

Mã	Mô tả yêu cầu
FR-01 Đăng nhập	Người dùng nhập username và password. Username phân biệt chữ hoa/thường. Nếu thành công, chuyển đến `inventory.html`. Nếu thất bại, hiển thị một trong các thông báo: `Username is required`, `Password is required`, `Username and password do not match any user in this service`, `Sorry, this user has been locked out.`
FR-02 Giỏ hàng	Mỗi sản phẩm có nút *Add to cart*/*Remove*. Badge trên biểu tượng giỏ hàng hiển thị số sản phẩm (1–6); khi giỏ rỗng badge không hiển thị.
FR-03 Thanh toán	First Name, Last Name, Postal Code đều bắt buộc, kiểm tra theo thứ tự đó. Trang tổng quan hiển thị Item total, Tax = 8% Item total (làm tròn 2 chữ số) và Total = Item total + Tax. Nhấn Finish hiển thị `Thank you for your order!` và làm rỗng giỏ hàng. Nhấn Cancel ở bước nhập thông tin quay về `cart.html`.
FR-04 Sắp xếp	Danh sách sản phẩm sắp xếp được theo Name (A to Z), Name (Z to A), Price (low to high), Price (high to low).
NFR-01 Hiệu năng	Sau khi nhấn Login với thông tin đúng, trang sản phẩm phải hiển thị trong tối đa 3 giây (trung vị của 5 lần đo, mạng ổn định).

4.2 Đánh giá rủi ro

Chấm *xác suất lỗi* (P) và *mức tác động* (I) theo thang 1–5, rủi ro $R = P \times I$. Yêu cầu có $R \geq 12$ phải được thiết kế nhiều test case nhất.

Mã	P	I	R	Lý do (ví dụ)
FR-03	3	5	15	Tính tiền sai gây thiệt hại trực tiếp; có nhiều nhánh nhập liệu.
FR-01	3	4	12	Có 4 loại thông báo lỗi, dễ hiển thị sai thông báo.
FR-04	2	2	4	Sắp xếp sai gây khó chịu nhưng không làm mất tiền.

4.3 Ma trận truy vết

Ma trận được lập ở NV2 và cập nhật dần đến cuối bài. Mã định danh phải giống nhau trong báo cáo, tệp Excel và `@DisplayName` của mã nguồn.

Yêu cầu	Kỹ thuật	Test case	Lớp test	Lỗi
FR-01	EP, DT	TC-LOGIN-EP1..EP5, TC-LOGIN-DT1..DT5	`LoginTest`	–
FR-02	BVA	TC-CART-B1..B5	`CartTest`	–
FR-03	UC	TC-CHK-01..05	`CheckoutTest`	–
NFR-01	Đo thời gian	TC-NFR-01	`PerformanceTest`	BUG-01

5 NV3 – Thiết kế test case

Mỗi kỹ thuật bên dưới có một ví dụ hoàn chỉnh. Nhóm được dùng lại ví dụ nhưng phải **tự áp dụng kỹ thuật đó cho ít nhất một trường/quy tắc khác**.

5.1 Phân lớp tương đương (EP)

Ví dụ: trường Username của FR-01 (password cố định là `secret_sauce`).

Lớp	Mô tả lớp	Giá trị đại diện	Kết quả mong đợi
EC1 (hợp lệ)	Tài khoản tồn tại, đang hoạt động	`standard_user`	Chuyển đến `inventory.html`
EC2 (không hợp lệ)	Tài khoản tồn tại nhưng bị khóa	`locked_out_user`	`Sorry, this user has been locked out.`
EC3 (không hợp lệ)	Chuỗi rỗng	`""`	`Username is required`
EC4 (không hợp lệ)	Tài khoản không tồn tại	`abc_user`	`Username and password do not match...`
EC5 (không hợp lệ)	Đúng tên nhưng sai chữ hoa/thường	`Standard_User`	`Username and password do not match...`

Mỗi lớp sinh một test case (TC-LOGIN-EP1 đến EP5). EC5 tồn tại vì đặc tả nói username phân biệt chữ hoa/thường; nếu bỏ lớp này, nhóm sẽ không phát hiện được hệ thống chấp nhận `STANDARD_USER`.

Nhóm tự làm thêm: EP cho trường Password hoặc Postal Code.

5.2 Phân tích giá trị biên (BVA)

Ví dụ: số sản phẩm trong giỏ hàng của FR-02. Miền hợp lệ là $[0, 6]$ vì cửa hàng có 6 sản phẩm.

Mã TC	Giá trị	Cách tạo	Kết quả mong đợi
TC-CART-B1	0 (min)	Đăng nhập, không thêm gì	Không có badge
TC-CART-B2	1 (min+1)	Thêm Sauce Labs Backpack	Badge = `1`
TC-CART-B3	5 (max−1)	Thêm đủ 6 rồi Remove 1	Badge = `5`
TC-CART-B4	6 (max)	Thêm cả 6 sản phẩm	Badge = `6`
TC-CART-B5	$1 \to 0$	Thêm 1 rồi Remove	Badge biến mất

Giá trị -1 và 7 không tạo được qua giao diện vì mỗi sản phẩm chỉ có một nút Add; nhóm ghi rõ "không áp dụng" kèm lý do này thay vì bỏ qua im lặng.

Nhóm tự làm thêm: BVA cho phép làm tròn thuế ở FR-03 (chọn tổ hợp sản phẩm sao cho phần thập phân của 8% nằm sát 0,005).

5.3 Bảng quyết định (DT)

Ví dụ: quy tắc đăng nhập của FR-01, gồm 4 điều kiện. Ký hiệu: Y = đúng, N = sai, – = không ảnh hưởng.

	R1	R2	R3	R4	R5
C1: Username rỗng	Y	N	N	N	N
C2: Password rỗng	–	Y	N	N	N
C3: Cặp username/password đúng	–	–	N	Y	Y
C4: Tài khoản bị khóa	–	–	–	Y	N
A1: `Username is required`	✓				
A2: `Password is required`		✓			
A3: `Username and password do not match...`			✓		
A4: `Sorry, this user has been locked out.`				✓	
A5: Chuyển đến `inventory.html`					✓

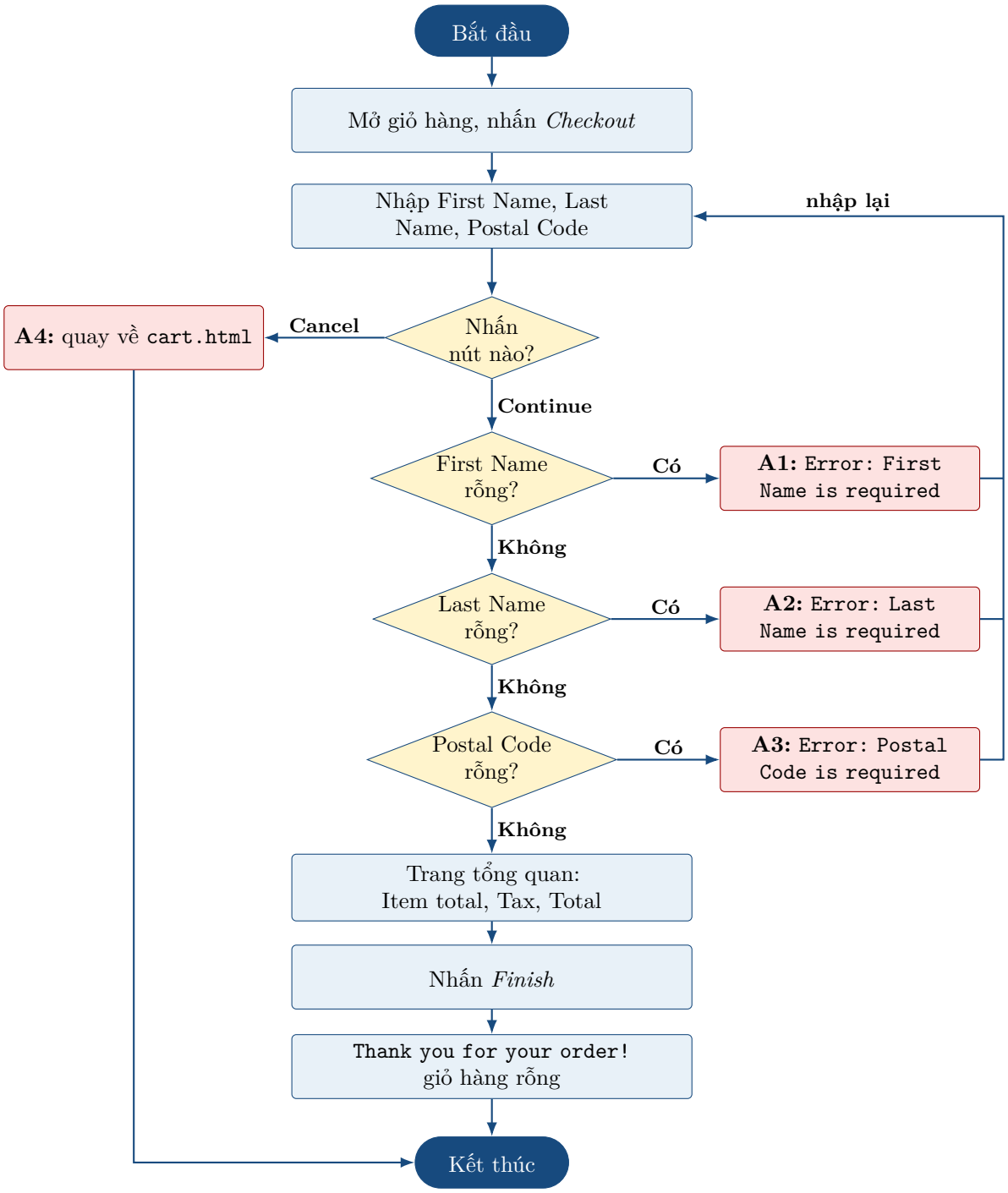
Quy tắc	Mã TC	Username	Password
R1	TC-LOGIN-DT1	`""`	`""`
R2	TC-LOGIN-DT2	`standard_user`	`""`
R3	TC-LOGIN-DT3	`locked_out_user`	`wrong_password`
R4	TC-LOGIN-DT4	`locked_out_user`	`secret_sauce`
R5	TC-LOGIN-DT5	`standard_user`	`secret_sauce`

Cách đọc bảng: R1 có C2 = "–" nghĩa là chỉ cần username rỗng thì luôn nhận A1, bất kể password. Để chứng minh điều này, nhóm nên chạy thêm `""/secret_sauce` và xác nhận vẫn nhận A1. Tương tự, R3 có C4 = "–": tài khoản bị khóa nhưng sai mật khẩu vẫn nhận A3, không phải A4. Mỗi quy tắc R1–R5 thành một test case TC-LOGIN-DT1 đến DT5.

Nhóm tự làm thêm: bảng quyết định cho bước nhập thông tin thanh toán (3 điều kiện: First Name, Last Name, Postal Code rỗng hay không).

5.4 Kiểm thử ca sử dụng (UC)

Ví dụ: ca sử dụng "Thanh toán đơn hàng" của FR-03. Tiền điều kiện: đã đăng nhập bằng `standard_user`, giỏ hàng có ít nhất một sản phẩm. Hình 2 là sơ đồ hoạt động; mỗi đường đi từ "Bắt đầu" đến "Kết thúc" là một kịch bản.



Hình 2: Sơ đồ hoạt động của ca sử dụng "Thanh toán đơn hàng". Ô đỏ là luồng thay thế/ngoại lệ; sau A1–A3 người dùng quay lại bước nhập thông tin.

Kịch bản rút ra từ sơ đồ:

Mã TC	Luồng	First / Last / Postal	Kết quả mong đợi
TC-CHK-01	Luồng chính	An / Nguyen / 700000	`Thank you for your order!`
TC-CHK-02	A1	"" / Nguyen / 700000	`Error: First Name is required`
TC-CHK-03	A2	An / "" / 700000	`Error: Last Name is required`
TC-CHK-04	A3	An / Nguyen / ""	`Error: Postal Code is required`
TC-CHK-05	A4	(không nhập) / Cancel	URL `cart.html`, sản phẩm vẫn còn trong giỏ

Đặc tả đầy đủ một test case. Mọi test case trong tệp Excel phải chi tiết tương đương:

Mã / Tên	TC-CHK-01 – Thanh toán thành công một sản phẩm
Yêu cầu / Kỹ thuật	FR-03 / Use case, luồng chính
Tiền điều kiện	Đã đăng nhập bằng `standard_user`; giỏ hàng rỗng
Dữ liệu	First Name = An, Last Name = Nguyen, Postal Code = 700000

Bước	Thao tác	Kết quả mong đợi
1	Nhấn *Add to cart* ở Sauce Labs Backpack	Badge giỏ hàng = `1`
2	Nhấn biểu tượng giỏ hàng	URL `cart.html`; có 1 sản phẩm giá $29.99
3	Nhấn *Checkout*	URL `checkout-step-one.html`
4	Nhập dữ liệu, nhấn *Continue*	URL `checkout-step-two.html`
5	Đọc phần tổng tiền	`Item total: $29.99`, `Tax: $2.40`, `Total: $32.39`
6	Nhấn *Finish*	URL `checkout-complete.html`; `Thank you for your order!`; không còn badge

Ở bước 5, kết quả mong đợi được tính từ đặc tả chứ không chép từ màn hình: $29{,}99 \times 8\% = 2{,}3992 \approx 2{,}40$ và $29{,}99 + 2{,}40 = 32{,}39$.

Nhóm tự làm thêm: sơ đồ hoạt động và kịch bản cho ca sử dụng thứ hai (gợi ý: "Quản lý giỏ hàng" với luồng thêm, xóa trên trang sản phẩm, xóa trong trang giỏ hàng, Continue Shopping).

5.5 Số lượng tối thiểu

Kỹ thuật	Số test case tối thiểu
EP (2 trường)	8
BVA (2 miền giá trị)	8
Bảng quyết định (2 bảng)	8
Use case (2 ca sử dụng, mỗi ca $\geq$ 2 luồng thay thế)	8
NFR-01	1

Test case trùng nhau giữa các kỹ thuật được dùng chung một mã và chỉ tính một lần.

6 NV4 – Thực thi thủ công và báo lỗi

6.1 Ghi kết quả thực thi

Mỗi lần chạy ghi một dòng. Trạng thái chỉ được là PASS, FAIL hoặc BLOCKED (không chạy được do môi trường). Các bảng trong mục này là ví dụ minh họa; số liệu trong bài nộp phải là số đo thật của nhóm.

Mã TC	Ngày / Người	Môi trường	Kết quả thực tế	Trạng thái
TC-CHK-01	07/10 – An	Chrome 154, macOS	Total: \$32.39; hiện Thank you	PASS
TC-NFR-01	07/10 – Bình	Chrome 154, macOS	Trung vị 5,1 s	FAIL (BUG-01)
TC-CART-B4	08/10 – Chi	Chrome 154, Win 11	Trang không tải được	BLOCKED

6.2 Báo cáo lỗi

Mã / Tiêu đề	BUG-01 – Trang sản phẩm mất khoảng 5 giây để hiển thị với `performance_glitch_user`
Mức nghiêm trọng	Major – vi phạm NFR-01, người dùng có thể tưởng hệ thống bị treo
Môi trường	Chrome 154, macOS, mạng cáp quang; ngày 07/10
Tiền điều kiện	Chưa đăng nhập, xóa cookie
Bước tái hiện	1. Mở `https://www.saucedemo.com` 2. Nhập `performance_glitch_user` / `secret_sauce` 3. Nhấn Login, bấm giờ đến khi danh sách sản phẩm hiện ra
Mong đợi	Danh sách sản phẩm hiển thị trong ≤ 3 s
Thực tế	5 lần đo: 5,2 – 5,1 – 5,1 – 5,3 – 5,1 s; trung vị 5,1 s
Bằng chứng	`evidence/BUG-01.mp4`, log của `PerformanceTest`

Một lỗi xuất hiện với nhiều bộ dữ liệu vẫn chỉ ghi *một* báo cáo; các test case liên quan cùng tham chiếu đến mã lỗi đó. Gợi ý: đăng nhập bằng `problem_user` và chạy lại TC-CHK-01 cùng các test case giỏ hàng – nhóm sẽ tìm được thêm lỗi chức năng.

7 NV5 – Tự động hóa bằng JUnit 5

7.1 Yêu cầu

- Tự động hóa ít nhất **15 test case** đã thiết kế ở NV3, trong đó có đủ TC-CHK-01..05.
- Ít nhất **3 phương thức `@ParameterizedTest`**, dùng ít nhất hai nguồn khác nhau trong số `@CsvFileSource`, `@CsvSource`, `@MethodSource`, `@ValueSource`.
- Mọi thao tác giao diện nằm trong lớp `pages/`; lớp test chỉ gọi Page Object và viết assertion.
- Chỉ dùng explicit wait (`WebDriverWait`); không dùng `Thread.sleep`.
- `@DisplayName` của mỗi test chứa mã test case để đối chiếu với ma trận truy vết.

7.2 Ví dụ Page Object

```java
public final class CheckoutPage {
    private static final By FIRST_NAME = By.id("first-name");
    private static final By LAST_NAME  = By.id("last-name");
    private static final By POSTAL     = By.id("postal-code");
    private static final By CONTINUE   = By.id("continue");
    private static final By ERROR      = By.cssSelector("[data-test='error']");
    private static final By TOTAL      = By.cssSelector(".summary_total_label");

    private final WebDriver driver;
    private final WebDriverWait wait;

    public CheckoutPage(WebDriver driver) {
        this.driver = driver;
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(8));
    }

    public CheckoutPage fillInformation(String first, String last, String postal) {
        wait.until(ExpectedConditions.visibilityOfElementLocated(FIRST_NAME))
            .sendKeys(first);
        driver.findElement(LAST_NAME).sendKeys(last);
        driver.findElement(POSTAL).sendKeys(postal);
        return this;
    }

    public String continueExpectingError() {
        driver.findElement(CONTINUE).click();
        return wait.until(ExpectedConditions.visibilityOfElementLocated(ERROR))
                   .getText();
    }

    public String continueToOverviewAndReadTotal() {
        driver.findElement(CONTINUE).click();
        return wait.until(ExpectedConditions.visibilityOfElementLocated(TOTAL))
                   .getText();
    }
}
```

Listing 2: `pages/CheckoutPage.java` (rút gọn)

7.3 Ví dụ test tham số hóa từ kịch bản use case

TC-CHK-02..04 có cùng các bước, chỉ khác dữ liệu và thông báo, nên được gộp thành một phương thức tham số hóa. Trong `@CsvSource`, `''` là chuỗi rỗng.

```java
@DisplayName("FR-03: Checkout")
class CheckoutTest extends WebTestBase {

    private CheckoutPage checkoutPage;

    @BeforeEach
    void goToCheckoutWithOneItem() {
        new LoginPage(driver, baseUrl).open().login("standard_user", "secret_sauce");
        new InventoryPage(driver).addToCart("sauce-labs-backpack").openCart();
        checkoutPage = new CartPage(driver).checkout();
```

```
    }

    @Test
    @DisplayName("TC-CHK-01: main flow computes total with 8% tax")
    void mainFlowShowsCorrectTotal() {
        String total = checkoutPage
                .fillInformation("An", "Nguyen", "700000")
                .continueToOverviewAndReadTotal();
        assertEquals("Total: $32.39", total);
    }

    @ParameterizedTest(name = "{0}: missing field -> {4}")
    @CsvSource({
        "TC-CHK-02, '', Nguyen, 700000, First Name is required",
        "TC-CHK-03, An, '',     700000, Last Name is required",
        "TC-CHK-04, An, Nguyen, '',     Postal Code is required"
    })
    void missingFieldShowsError(String id, String first, String last,
                                String postal, String expected) {
        String message = checkoutPage
                .fillInformation(first, last, postal)
                .continueExpectingError();
        assertTrue(message.contains(expected),
                () -> id + " got: " + message);
    }
}
```

Listing 3: `tests/CheckoutTest.java` (rút gọn)

7.4 Ví dụ kiểm tra sắp xếp (FR-04)

Kết quả mong đợi được tính bằng cách tự sắp xếp danh sách giá đọc được, không ghi cứng thứ tự.

```
@ParameterizedTest(name = "TC-SORT-{index}: option={0}")
@CsvSource({ "lohi, true", "hilo, false" })
void sortByPrice(String option, boolean ascending) {
    List<Double> actual = inventoryPage.sortBy(option).prices();
    List<Double> expected = new ArrayList<>(actual);
    expected.sort(ascending ? Comparator.naturalOrder()
                            : Comparator.reverseOrder());
    assertEquals(expected, actual);
}
```

7.5 Ví dụ đo NFR-01 và đánh dấu lỗi đã biết

```
@Test
@Tag("known-bug")
@DisplayName("TC-NFR-01 [BUG-01]: inventory loads within 3 s")
void inventoryLoadsWithinThreeSeconds() {
    long start = System.nanoTime();
    LoginResult result = loginPage.login("performance_glitch_user", "secret_sauce");
    long millis = Duration.ofNanos(System.nanoTime() - start).toMillis();

    assertTrue(result.accepted());
    assertTrue(millis <= 3000, () -> "Inventory took " + millis + " ms");
```

```
}
```

Test này *phải* FAIL vì hệ thống có lỗi. Không nới ngưỡng lên 6 giây để test PASS. Có thể tách nhóm lỗi đã biết khi chạy:

```
mvn test -DexcludedGroups=known-bug    # chi chay test khong co loi da biet
mvn test -Dgroups=known-bug            # chi chay test lien quan den BUG
```

7.6 Chờ đúng cách

```
// Sai: cho co dinh, cham khi trang nhanh va van loi khi trang cham
Thread.sleep(3000);
driver.findElement(By.cssSelector(".shopping_cart_badge")).getText();

// Dung: cho den khi dieu kien quan sat duoc xay ra
wait.until(ExpectedConditions.textToBe(
        By.cssSelector(".shopping_cart_badge"), "1"));
```

7.7 Điểm bonus

- **Data-driven mức 2**: URL, locator và dữ liệu của một kịch bản được đọc từ tệp cấu hình (ví dụ `checkout-flow.csv` chứa cột `page`, `locator`, `action`, `value`); đổi sang website khác chỉ cần sửa tệp, không sửa Java.
- Chạy bộ test trên GitHub Actions hoặc GitLab CI, lưu `surefire-reports` làm artifact.
- JUnit 5 extension (`TestWatcher`) tự chụp ảnh màn hình khi test FAIL.

8 Sản phẩm nộp

```
group-XX-assignment1/
|-- report.pdf                    # bao cao NV1 - NV5
|-- test-cases.xlsx               # sheet: Requirements, Traceability,
|                                 #   TestCases, Execution, Bugs
|-- evidence/                     # anh, video, dat ten theo ma TC/BUG
`-- automation/                   # du an Maven, KHONG nop thu muc target/
    |-- pom.xml
    |-- README.md                 # lenh chay, phien ban Java/trinh duyet
    `-- src/test/...
```

9 Tự kiểm tra trước khi nộp

- ☐ `mvn clean test` chạy được từ thư mục vừa giải nén; chỉ các test có `@Tag("known-bug")` được phép FAIL.
- ☐ Mỗi yêu cầu FR/NFR có ít nhất một test case; mỗi test case FAIL có mã BUG.
- ☐ Kết quả mong đợi được tính từ đặc tả, không chép từ màn hình sau khi chạy.
- ☐ Mã test case trong Excel, báo cáo và `@DisplayName` trùng nhau.
- ☐ Không có `Thread.sleep` trong mã nguồn.